

Crie um sistema de atributos para um simulador de manager de MOBA, inspirado no modelo de atributos de Dota 2, mas implementado como sistema próprio e configurável.

O objetivo é permitir que cada herói tenha atributos-base, crescimento por nível, atributos derivados, ataque, defesa, resistências, movimento, visão, funções táticas e escalonamento por atributo primário.

Não implementar nenhuma parte relacionada a animações, attack point, attack backswing, cast point ou cast backswing.

Use nomes de código em inglês, mas mantenha comentários e documentação em português quando útil.

1. Conceitos principais

Cada herói deve ter três atributos-base numéricos:

- strength
- agility
- intelligence

Cada herói também deve ter um tipo de atributo primário:

- strength
- agility
- intelligence
- universal

O atributo primário define como o herói ganha dano de ataque a partir dos atributos.

Heróis de strength ganham dano principalmente com strength.

Heróis de agility ganham dano principalmente com agility.

Heróis de intelligence ganham dano principalmente com intelligence.

Heróis universal ganham dano a partir da soma dos três atributos, mas com coeficiente menor por ponto.

2. Constantes globais configuráveis

Crie um arquivo de constantes para facilitar o balanceamento do jogo.

Valores iniciais sugeridos:

```
export const ATTRIBUTE_RULES = {  
  healthPerStrength: 22,  
  healthRegenPerStrength: 0.1,
```

```

armorPerAgility: 1 / 6,
attackSpeedPerAgility: 1,

manaPerIntelligence: 12,
manaRegenPerIntelligence: 0.05,
magicResistancePerIntelligence: 0.1,

primaryAttributeDamageMultiplier: 1,
universalAttributeDamageMultiplier: 0.7,
};

```

Esses valores devem ser fáceis de alterar depois para balanceamento.

3. Modelo de dados do herói

Criar uma interface ou tipo HeroDefinition.

Campos obrigatórios:

```

export type PrimaryAttribute =
  | "strength"
  | "agility"
  | "intelligence"
  | "universal";

export type AttackType = "melee" | "ranged";

export type HeroRole =
  | "carry"
  | "support"
  | "nuker"
  | "disabler"
  | "durable"
  | "escape"
  | "pusher"
  | "initiator";

export interface AttributeBlock {
  strength: number;
  agility: number;
  intelligence: number;
}

export interface AttributeGrowthBlock {
  strengthGain: number;
  agilityGain: number;
  intelligenceGain: number;
}

```

```

export interface HeroDefinition {
  id: string;
  name: string;

  primaryAttribute: PrimaryAttribute;
  attackType: AttackType;
  roles: HeroRole[];

  complexity: 1 | 2 | 3;

  baseAttributes: AttributeBlock;
  attributeGrowth: AttributeGrowthBlock;

  baseStats: {
    baseHealth: number;
    baseHealthRegen: number;

    baseMana: number;
    baseManaRegen: number;

    baseDamageMin: number;
    baseDamageMax: number;

    baseArmor: number;
    baseMagicResistance: number;
    baseStatusResistance: number;
    baseSlowResistance: number;

    baseAttackSpeed: number;
    baseAttackTime: number;
    attackRange: number;
    acquisitionRange: number;

    movementSpeed: number;
    turnRate: number;
    collisionSize: number;

    dayVision: number;
    nightVision: number;
  };
}

```

4. Estatísticas calculadas

Criar uma interface HeroCalculatedStats.

Ela representa a ficha final do herói em determinado nível, depois de aplicar atributos, itens, buffs e modificadores.

```

export interface HeroCalculatedStats {
  level: number;

  attributes: {
    strength: number;
    agility: number;
    intelligence: number;
    totalAttributes: number;
  };

  resources: {
    maxHealth: number;
    healthRegen: number;
    maxMana: number;
    manaRegen: number;
  };

  offense: {
    damageMin: number;
    damageMax: number;
    averageDamage: number;

    attackSpeed: number;
    baseAttackTime: number;
    attacksPerSecond: number;

    attackType: AttackType;
    attackRange: number;
    acquisitionRange: number;
  };

  defense: {
    armor: number;
    physicalDamageReduction: number;

    magicResistance: number;
    statusResistance: number;
    slowResistance: number;

    evasion: number;
    damageBlock: number;
  };

  movement: {
    movementSpeed: number;
    turnRate: number;
    collisionSize: number;
  };

  vision: {

```

```
    dayVision: number;  
    nightVision: number;  
  };  
}
```

5. Cálculo de atributos por nível

Criar uma função:

```
calculateAttributesAtLevel(hero: HeroDefinition, level: number):  
AttributeBlock
```

Regra:

- Nível mínimo: 1
- Nível máximo configurável, inicialmente 30
- No nível 1, usar baseAttributes
- Para cada nível acima do 1, adicionar o ganho correspondente

Fórmula:

```
strength = baseStrength + strengthGain * (level - 1)  
agility = baseAgility + agilityGain * (level - 1)  
intelligence = baseIntelligence + intelligenceGain * (level - 1)
```

6. Cálculo de dano por atributo primário

Criar função:

```
calculateAttributeDamage(  
  primaryAttribute: PrimaryAttribute,  
  attributes: AttributeBlock  
): number
```

Regras:

Para herói de strength:

```
damage = strength * primaryAttributeDamageMultiplier
```

Para herói de agility:

```
damage = agility * primaryAttributeDamageMultiplier
```

Para herói de intelligence:

```
damage = intelligence * primaryAttributeDamageMultiplier
```

Para herói universal:

```
damage = (strength + agility + intelligence) *  
universalAttributeDamageMultiplier
```

7. Cálculo de recursos

A vida máxima deve ser:

```
maxHealth = baseHealth + strength * healthPerStrength
```

A regeneração de vida deve ser:

```
healthRegen = baseHealthRegen + strength * healthRegenPerStrength
```

A mana máxima deve ser:

```
maxMana = baseMana + intelligence * manaPerIntelligence
```

A regeneração de mana deve ser:

```
manaRegen = baseManaRegen + intelligence * manaRegenPerIntelligence
```

8. Cálculo de ataque

O dano mínimo deve ser:

```
damageMin = baseDamageMin + attributeDamage
```

O dano máximo deve ser:

```
damageMax = baseDamageMax + attributeDamage
```

O dano médio deve ser:

```
averageDamage = (damageMin + damageMax) / 2
```

A velocidade de ataque deve ser:

```
attackSpeed = baseAttackSpeed + agility * attackSpeedPerAgility
```

Os ataques por segundo devem ser calculados usando:

```
attacksPerSecond = attackSpeed / (100 * baseAttackTime)
```

Aplicar limites configuráveis:

```
minAttackSpeed = 20  
maxAttackSpeed = 700
```

Antes de calcular ataques por segundo, limitar attackSpeed entre minAttackSpeed e maxAttackSpeed.

9. Cálculo de defesa

A armadura deve ser:

```
armor = baseArmor + agility * armorPerAgility
```

A redução de dano físico deve usar a fórmula:

```
physicalDamageReduction = (0.06 * armor) / (1 + 0.06 * Math.abs(armor))
```

Essa fórmula deve aceitar armadura negativa.

A resistência mágica deve ser:

```
magicResistance =  
    baseMagicResistance + intelligence * magicResistancePerIntelligence
```

A resistência mágica deve ser tratada como porcentagem.

Exemplo:

```
25 = 25%
```

A resistência de status deve começar com:

```
statusResistance = baseStatusResistance
```

A resistência a lentidão deve começar com:

```
slowResistance = baseSlowResistance
```

Evasão e bloqueio de dano devem começar em zero, salvo se itens, talentos ou habilidades adicionarem valores.

```
evasion = 0  
damageBlock = 0
```

10. Movimento

Cada herói deve ter:

```
movementSpeed  
turnRate  
collisionSize
```

Aplicar limites configuráveis para velocidade de movimento:

```
minMovementSpeed = 100  
maxMovementSpeed = 550
```

A velocidade final deve ser limitada entre esses valores, salvo quando algum modificador especial permitir ultrapassar o limite.

11. Visão

Cada herói deve ter:

```
dayVision  
nightVision
```

Valores padrão sugeridos:

```
dayVision = 1800  
nightVision = 800
```

Permitir exceções por herói.

12. Modificadores

Criar um sistema de modificadores para itens, buffs, debuffs, talentos e efeitos de treinamento.

Um modificador deve poder afetar:

- atributos-base
- vida máxima
- regeneração de vida
- mana máxima
- regeneração de mana
- dano
- armadura
- resistência mágica
- resistência de status
- resistência a lentidão
- evasão
- bloqueio de dano
- velocidade de ataque
- base attack time
- alcance de ataque
- velocidade de movimento
- visão diurna
- visão noturna

Criar tipo:

```
export interface StatModifier {  
  id: string;  
  source: "item" | "buff" | "debuff" | "talent" | "training" | "aura";  
  
  flat?: Partial<{  
    strength: number;  
    agility: number;  
    intelligence: number;  
  
    maxHealth: number;  
    healthRegen: number;  
    maxMana: number;  
    manaRegen: number;  
  
    damageMin: number;  
    damageMax: number;  
  
    armor: number;  
    magicResistance: number;  
    statusResistance: number;  
    slowResistance: number;  
  
    evasion: number;
```

```

    damageBlock: number;

    attackSpeed: number;
    baseAttackTime: number;
    attackRange: number;
    acquisitionRange: number;

    movementSpeed: number;
    turnRate: number;
    collisionSize: number;

    dayVision: number;
    nightVision: number;
  }>;

  percent?: Partial<{
    maxHealth: number;
    healthRegen: number;
    maxMana: number;
    manaRegen: number;

    damage: number;
    armor: number;
    magicResistance: number;

    attackSpeed: number;
    movementSpeed: number;

    dayVision: number;
    nightVision: number;
  }>;
}

```

13. Ordem de cálculo recomendada

A função principal deve seguir esta ordem:

```
calculateHeroStats(hero, level, modifiers)
```

Ordem:

1. Validar nível
2. Calcular atributos por nível
3. Aplicar modificadores flat de atributos
4. Calcular recursos derivados dos atributos
5. Calcular dano derivado do atributo primário
6. Calcular ataque
7. Calcular defesa
8. Calcular movimento

9. Calcular visão
10. Aplicar modificadores flat em estatísticas derivadas
11. Aplicar modificadores percentuais
12. Aplicar limites mínimos e máximos
13. Retornar HeroCalculatedStats

14. Exemplo de herói

Criar um herói de exemplo para teste:

```
export const exampleHero: HeroDefinition = {
  id: "iron_vanguard",
  name: "Iron Vanguard",

  primaryAttribute: "strength",
  attackType: "melee",
  roles: ["durable", "initiator"],
  complexity: 1,

  baseAttributes: {
    strength: 25,
    agility: 14,
    intelligence: 16,
  },

  attributeGrowth: {
    strengthGain: 3.2,
    agilityGain: 1.6,
    intelligenceGain: 1.8,
  },

  baseStats: {
    baseHealth: 120,
    baseHealthRegen: 1.5,

    baseMana: 75,
    baseManaRegen: 0,

    baseDamageMin: 30,
    baseDamageMax: 36,

    baseArmor: 1,
    baseMagicResistance: 25,
    baseStatusResistance: 0,
    baseSlowResistance: 0,

    baseAttackSpeed: 100,
    baseAttackTime: 1.7,
    attackRange: 150,
    acquisitionRange: 600,
```

```
movementSpeed: 300,  
turnRate: 0.6,  
collisionSize: 24,  
  
dayVision: 1800,  
nightVision: 800,  
},  
};
```

15. Testes obrigatórios

Criar testes unitários para:

1. Calcular atributos corretamente no nível 1.
2. Calcular atributos corretamente em nível acima de 1.
3. Herói de strength ganhar dano com strength.
4. Herói de agility ganhar dano com agility.
5. Herói de intelligence ganhar dano com intelligence.
6. Herói universal ganhar dano com a soma dos três atributos.
7. Strength aumentar vida máxima e regeneração de vida.
8. Agility aumentar armadura e velocidade de ataque.
9. Intelligence aumentar mana, regeneração de mana e resistência mágica.
10. Armadura positiva reduzir dano físico.
11. Armadura negativa aumentar dano físico recebido.
12. Velocidade de ataque respeitar limites mínimo e máximo.
13. Velocidade de movimento respeitar limites mínimo e máximo.
14. Modificadores flat serem aplicados corretamente.
15. Modificadores percentuais serem aplicados depois dos flats.
16. A função principal retornar uma ficha completa do herói.

16. Resultado esperado

Ao final, o projeto deve permitir:

```
const stats = calculateHeroStats(exampleHero, 10, []);
```

E retornar uma ficha completa com:

- atributos finais
- vida
- regeneração de vida
- mana
- regeneração de mana
- dano mínimo
- dano máximo
- dano médio
- armadura
- redução física

- resistência mágica
- resistência de status
- resistência a lentidão
- evasão
- bloqueio de dano
- velocidade de ataque
- ataques por segundo
- tipo de ataque
- alcance de ataque
- velocidade de movimento
- turn rate
- tamanho de colisão
- visão diurna
- visão noturna

Não incluir nenhuma estatística de animação.

Não implementar habilidades ainda.

Não implementar combate completo ainda.

O foco desta etapa é criar o sistema matemático central de atributos e estatísticas de heróis para servir como fundação do simulador de manager de MOBA.